

프로젝트 한눈에 보기

서비스 목표

영화 정보와 리뷰를 관리하고, 리뷰 감성 분석 결과를 평균 평점으로 시각화하는 웹 서비스를 구현했다.

구현 방식

프론트는 Streamlit, 백엔드는 FastAPI로 분리하고, 데이터는 SQLite에 저장했다.

감성 분석

KoELECTRA 기반 분류 모델을 FastAPI 내부에서 직접 호출해 라벨과 점수를 생성했다.

시연 준비

영화 3개와 영화별 리뷰 10개 이상을 시드 스크립트로 재현 가능하게 구성했다.

기술 스택

- Frontend: Streamlit
- Backend: FastAPI
- Database: SQLite / SQLAlchemy
- Model: Copycats/koelectra-base-v3-generalized-sentiment-analysis
- Environment: uv

1. 서비스 개요

- 팀/이름: 1팀 안호성
- 서비스 형태:
 - Streamlit 프론트엔드 + FastAPI 백엔드 기반 영화 리뷰 감성 분석 웹 애플리케이션
- 서비스 목표:
 - 영화 정보를 등록하고 목록으로 관리
 - 영화별 리뷰를 저장하고 최근 리뷰를 확인
 - 리뷰 등록 시 감성 분석을 자동 수행하고 평균 평점으로 시각화

이번 미션에서는 영화 정보와 사용자 리뷰를 함께 관리하고, 등록된 리뷰에 대해 자동으로 감성 분석을 수행하는 웹 애플리케이션을 구현했다. 사용자는 영화 제목, 개봉일, 감독, 장르, 포스터 URL을 등록할 수 있으며, 각 영화에 대해 리뷰를 작성할 수 있다. 리뷰가 등록되면 백엔드에서 감성 분석 모델을 실행해 감성 라벨과 감성 점수를 저장하고, 이 결과를 바탕으로 영화별 평균 평점을 계산해 화면에 표시하도록 구성했다.

2. 제출 산출물

- 프론트엔드 코드:
 - frontend/
- 백엔드 코드:
 - backend/
- 구조 및 ERD 문서:
 - mission18_architecture_and_erd.md
- 캡처 체크리스트:
 - mission18_capture_checklist.md
- 요약 보고서:
 - 미션18_1팀_안호성_요약보고서.md
- 캡처 이미지:
 - backend01.png
 - backend02.png
 - backend03.png
 - frontend01.png
 - frontend02.png

3. 서비스 구조

본 서비스는 프론트엔드와 백엔드를 분리한 구조로 설계했다. 사용자는 Streamlit 화면에서 영화와 리뷰를 입력하고, 프론트엔드는 이를 FastAPI 백엔드에 HTTP 요청으로 전달한다. 백엔드는 SQLite에 데이터를 저장하고, 리뷰 등록 시 감성 분석 모델을 호출해 감성 결과를 함께 기록한다.

사용자 → Streamlit 프론트엔드 → FastAPI 백엔드로 요청이 전달되며, 백엔드는 SQLite 데이터베이스와 감성 분석 모델을 함께 사용해 결과를 반환한다.

이 구조를 통해 프론트엔드는 화면 구성과 사용자 입력에 집중하고, 백엔드는 데이터 저장과 비즈니스 로직 처리에 집중할 수 있도록 했다.

4. 기술 스택 및 구현 방식

- 프론트엔드:
 - Streamlit
- 백엔드:
 - FastAPI
- 데이터베이스:
 - SQLite
- ORM:
 - SQLAlchemy
- 요청/응답 검증:
 - Pydantic
- 감성 분석:
 - Transformers
 - 모델: Copypcats/koelectra-base-v3-generalized-sentiment-analysis
- 실행 환경:
 - uv

구현 방식은 다음과 같다.

- 프론트엔드는 자체 저장 기능을 두지 않고 모든 데이터를 백엔드 API를 통해 관리했다.
- 백엔드는 영화와 리뷰를 각각 테이블로 분리하여 관리했다.
- 감성 분석 모델은 별도의 독립 서버 없이 FastAPI 내부에서 직접 로드하여 사용했다.
- 리뷰 등록 직후 `st.rerun()`을 사용해 평균 평점과 최근 리뷰가 즉시 갱신되도록 구성했다

5. 주요 기능 구현 결과

이번 미션에서 구현한 주요 기능은 다음과 같다.

- 영화 등록
- 전체 영화 목록 조회
- 특정 영화 조회
- 특정 영화 삭제
- 리뷰 등록
- 전체 리뷰 조회
- 특정 영화 리뷰 조회
- 리뷰 삭제
- 리뷰 등록 시 감성 분석 자동 수행
- 영화별 평균 평점 조회
- 최근 10개 리뷰 표시

구현 결과, 영화 카드에서 포스터, 감독, 장르, 개봉일, 평균 평점을 함께 확인할 수 있도록 했고, 리뷰를 등록하면 감성 결과와 감성 점수가 저장되며 평균 평점에도 즉시 반영 되도록 구성했다.

6. 데이터베이스 구조도(ERD)

서비스의 핵심 데이터는 Movie 테이블과 Review 테이블로 구성된다.

- Movie
 - id: INTEGER PK
 - title: VARCHAR(200) UNIQUE
 - release_date: DATE
 - director: VARCHAR(100)
 - genre: VARCHAR(100)
 - poster_url: VARCHAR(500)
 - created_at: DATETIME
- Review
 - id: INTEGER PK
 - movie_id: INTEGER FK -> Movie.id
 - author: VARCHAR(100)
 - content: TEXT
 - sentiment_label: VARCHAR(20)
 - sentiment_score: FLOAT
 - created_at: DATETIME

관계 설명:

- Movie 1 : N Review
- 하나의 영화는 여러 개의 리뷰를 가진다.
- 하나의 리뷰는 하나의 영화에만 속한다.

이 구조를 통해 영화별 리뷰 조회와 평균 평점 계산을 자연스럽게 처리할 수 있도록 했다.

7. 평균 평점 계산 방식

감성 분석 결과는 0에서 1 사이의 점수로 정규화해 저장했다. 영화별 평균 평점은 이 sentiment_score 평균을 계산한 뒤 5점 만점 기준으로 환산하는 방식으로 구현했다.

평균 평점은 평균 감성 점수에 5를 곱하는 방식으로 계산했다.

이 방식을 적용함으로써 감성 분석 결과를 사용자가 보다 직관적으로 이해할 수 있는 평점 형태로 제공할 수 있었다.

8. 감성 분석 모델 및 서빙 방식

감성 분석에는 Copycats/koelectra-base-v3-generalized-sentiment-analysis 모델을 사용했다. 이 모델은 한국어 감성 분류에 적합하며, 대형 생성형 모델에 비해 가볍게 실행할 수 있다는 장점이 있다.

이번 프로젝트에서는 별도의 모델 서버를 두지 않고 FastAPI 내부에서 감성 분석 모델을 직접 로드하여 추론하는 방식을 선택했다. 과제 규모와 시연 목적을 고려했을 때, 복잡한 서빙 인프라보다 구조의 단순성과 실행 안정성을 우선하는 것이 더 적절하다고 판단했다.

선택 이유는 다음과 같다.

- 한국어 감성 분류에 적합
- 생성형 모델보다 가볍게 실행 가능
- 제출 및 시연 환경에서 관리가 단순함
- 리뷰 등록 시 즉시 추론하기에 충분한 성능 제공

9. 샘플 데이터 및 검증 결과

시연을 위해 영화 3편과 각 영화당 10개 이상의 리뷰를 입력할 수 있도록 시드 스크립트를 작성했다.

- 샘플 영화:

- 기생충
- 올드보이
- 인터스텔라

- 샘플 리뷰:

- 각 영화별 10개 이상

포스터 URL은 실제로 접근 가능한 링크만 사용하도록 검증 후 반영했다. 초기 Wikimedia 한국어 경로 중 일부는 404를 반환했기 때문에, 최종적으로는 English Wikipedia 페이지의 og:image 링크를 사용했다.

검증 완료 링크:

- 기생충:

- https://upload.wikimedia.org/wikipedia/en/5/53/Parasite_%282019_film%29.png

- 올드보이:

- <https://upload.wikimedia.org/wikipedia/en/6/67/Oldboykoreanposter.jpg>

- 인터스텔라:

- https://upload.wikimedia.org/wikipedia/en/b/bc/Interstellar_film_poster.jpg

10. 실행 방법

- 백엔드 실행:

```
cd backend
uv venv
source .venv/bin/activate
uv pip install -r requirements.txt
uv run uvicorn app.main:app --reload
```

- 프론트엔드 실행:

```
cd frontend
uv venv
source .venv/bin/activate
uv pip install -r requirements.txt
uv run streamlit run app.py
```

- 샘플 데이터 입력:

```
cd backend
uv run python scripts/seed_sample_data.py
uv run python scripts/seed_reviews.py
```

- API 문서 접속:

- Swagger UI: <http://127.0.0.1:8000/docs>
- ReDoc: <http://127.0.0.1:8000/redoc>

11. 캡처 자료 정리

제출 폴더에는 서비스 동작과 FastAPI Docs 관련 캡처 이미지를 함께 정리했다.

- backend01.png
- backend02.png
- backend03.png
- frontend01.png
- frontend02.png

이 이미지를 통해 다음 항목을 제출 자료에 포함할 수 있다.

- FastAPI Docs 전체 캡처
- 서비스 동작 캡처
- 영화 목록 및 포스터 표시 화면
- 리뷰 등록 및 감성 분석 결과 화면
- 평균 평점 표시 화면

12. 결론

이번 미션에서는 Streamlit과 FastAPI를 조합해 영화 리뷰 감성 분석 웹 애플리케이션을 구현했다. 프론트엔드와 백엔드를 분리한 구조를 적용함으로써 화면 구성과 데이터 처리 책임을 명확하게 구분할 수 있었고, SQLite와 SQLAlchemy를 통해 과제 제출에 적합한 단순한 데이터 관리 구조를 구성할 수 있었다.

또한 리뷰 등록 직후 감성 분석 결과와 평균 평점이 즉시 반영되도록 구성해, 단순한 CRUD 기능을 넘어 사용자에게 즉각적인 피드백을 제공하는 형태의 서비스를 완성했다. 결과적으로 과제에서 요구한 영화 등록, 리뷰 등록, 감성 분석, 평균 평점 표시, API 문서화, 캡처 자료 정리를 모두 충족하는 형태로 마무리했다.

FastAPI Docs 및 백엔드 캡처

backend01.png

Mission 18 Movie Review API 1.0.0 QAS 3.1
/openapi.json

영화 정보 등록, 리뷰 등록, 감상 분석, 평균 평점 조회를 담당하는 FastAPI 백엔드입니다.

Health

- GET** / Root
- GET** /health Health Check

Movies

- GET** /movies 영화 전체 조회
- POST** /movies 영화 등록
- GET** /movies/{movie_id} 특정 영화 조회
- DELETE** /movies/{movie_id} 영화 삭제
- GET** /movies/{movie_id}/rating 평균 평점 조회

Reviews

- GET** /movies/{movie_id}/reviews 특정 영화 리뷰 조회
- GET** /reviews 리뷰 전체 조회

backend02.png

Reviews

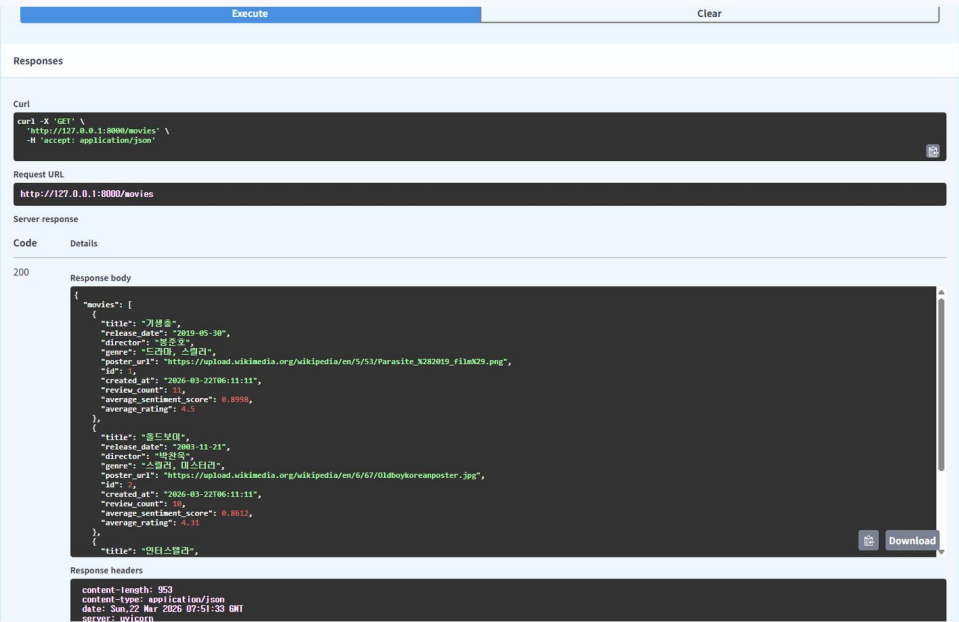
- GET** /movies/{movie_id}/reviews 특정 영화 리뷰 조회
- GET** /reviews 리뷰 전체 조회
- POST** /reviews 리뷰 등록 및 감상 분석
- DELETE** /reviews/{review_id} 리뷰 삭제

Schemas

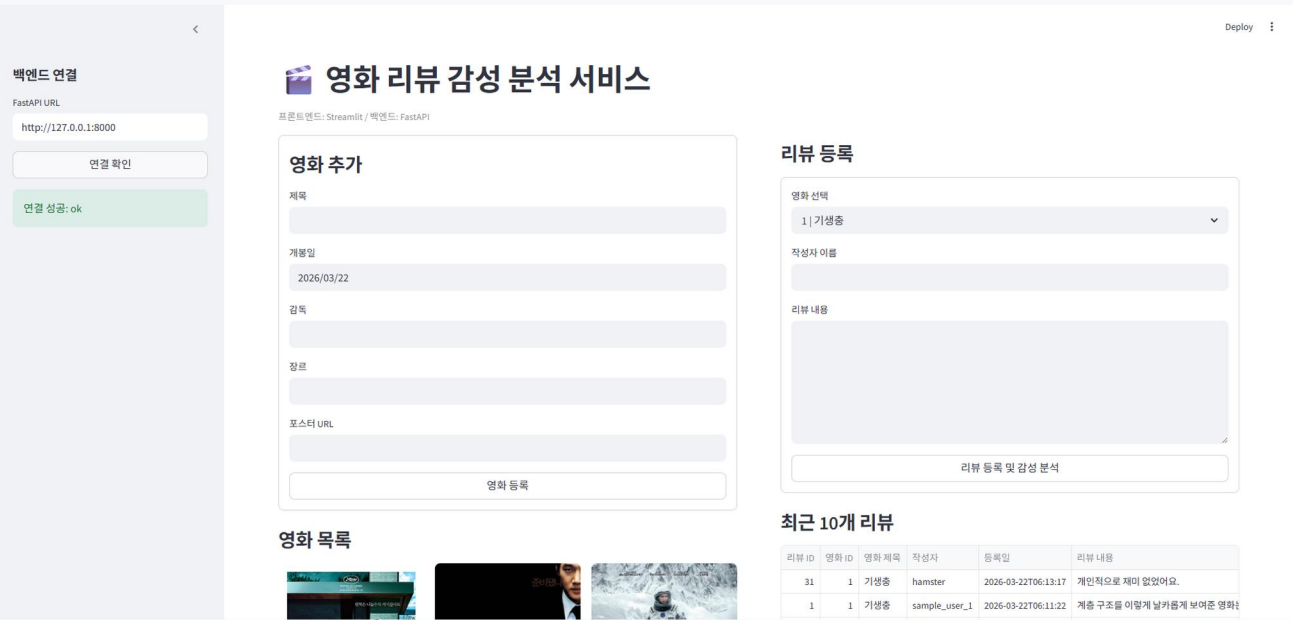
- AverageRatingResponse** > Expand all **object**
- DeleteResponse** > Expand all **object**
- HTTPValidationError** > Expand all **object**
- MovieCreate** > Expand all **object**
- MovieListResponse** > Expand all **object**
- MovieResponse** > Expand all **object**
- ReviewCreate** > Expand all **object**
- ReviewCreateResponse** > Expand all **object**

서비스 동작 캡처

backend03.png



frontend01.png



프론트엔드 화면 캡처

